Agentic Synthesis against Counterexample-Supplemented Sketches

Muness Castle Eric Rubeck

July 1, 2026

Abstract

Coding agents synthesize useful patches and can lose track of intent. A prompt can describe what to build and still leave unstated which plausible implementation would violate policy. We present *agentic synthesis against counterexample-supplemented sketches*, a method for programming with agents under checkable rules. A human writes a partial program-like sketch that fixes strategy and names holes; a finite corpus stores observations and promoted counterexamples; subject-matter experts correct concrete outputs; an agent edits code and prompts; and a gate refuses completion until replay and semantic comparison pass every promoted case.

The method imports the loop grammar of sketching and counterexample-guided inductive synthesis (CEGIS) and changes the synthesizer. In classical Sketch, a solver fills holes inside a formal language. Here, the synthesizer is an IDE-shaped coding agent whose edits span ordinary source code and prompt surfaces. The guarantee is finite-corpus correctness: when the gate passes, the artifact is correct for the promoted corpus under the replay and comparison semantics encoded by the repository.

The method came from a proprietary enterprise harness: a custom Cursor extension, `cursor://`-style commands, and an embedded web application through which subject-matter experts loaded production examples, corrected outputs, and helped turn those corrections into input/output specifications for the counterexample loop. The public companion repository gives readers a clean RosterSynth example of the same control structure. The paper develops the technique; the repository supplies runnable evidence.

1 Introduction

A coding agent can produce a plausible patch before the domain rule has been made explicit. That is the central risk this paper addresses. The agent sees a task, a codebase, a prompt, and perhaps a failing test. It can then synthesize the smallest local change that turns the visible signal green. The human question is different: did the patch preserve the policy the prompt failed to spell out?

Conventional tests catch many failures. They also leave gaps. A test can say that output shape is valid, an exception disappeared, or a state delta closed. The test rarely says which nearby repair would have been tempting and wrong, which rule the failure exposed, or how a future agent should avoid repeating the same mistake. Natural-language prompts have the opposite problem. They can hold the intended rule, yet they drift across sessions and forks. A repository needs a stable artifact that carries both the human strategy and the examples that sharpen it.

This paper describes that artifact and the loop around it. The ingredients are:

1. a **sketch** $\mathcal{S}$: a partial program in prose and code-shaped structure;
2. a **counterexample-supplemented corpus** $\mathcal{E}$: examples, observations, and promoted failures;
3. **known-code anchors** $\mathcal{K}$: codebase-local shapes, APIs, conventions, and reference paths the agent must preserve;

4. a **dual-oracle implementation surface**: deterministic code for encodable holes and prompt-mediated completion for narrative holes;

5. a **gate** $\mathcal{G}$: replay plus semantic comparison over every promoted case.

The sketch is the human's strategic claim. The corpus is the record of cases that have earned authority. The gate is the boundary of done. A failure becomes valuable when it changes the sketch and the corpus so future agent passes inherit the rule.

Central claim. A coding-agent workflow becomes inspectable when every promoted counterexample connects four things: the sketch clause it supplements, the implementation or prompt surface that handles it, the replay check that verifies state repair, and the semantic compare check that rejects the tempting wrong repair. This claim is bounded. It gives maintainers a theorem over the current promoted corpus and a procedure for growing that corpus when new failures appear.

Contributions. This paper makes five contributions.

1. It defines counterexample-supplemented sketches as a repository-native artifact for agentic coding.

2. It gives a process model for subject-matter expert corrections becoming input/output specs, counterexamples, and golden data.

3. It separates two hole-filling roles: Oracle A, deterministic code the agent maintains, and Oracle B, prompt/LLM/cassette behavior for sketch-declared narrative holes.

4. It states and proves a finite-corpus correctness theorem for replay/compare gates.

5. It gives a sketch-level RosterSynth example and a companion repository for readers who prefer runnable evidence.

The sections proceed in the order a team needs to understand the method. We start with the enterprise origin and the specification problem, place the method in sketching and CEGIS lineage, define the artifacts and roles, then describe the counterexample lifecycle, gate semantics, bounded proof, RosterSynth sketch, adoption workflow, companion artifact, discussion, and limitations.

2 Originating setting

The method came from a proprietary enterprise deployment. The work involved heterogeneous operational data, multiple downstream clients with their own rules, and subject-matter experts who understood the data far better than the code. The harness let those experts operate the counterexample loop through a review interface; agents edited code and prompts.

The deployed system joined six pieces:

1. a custom Cursor extension for invoking repository actions from the IDE;

2. `cursor://`-style URL commands connecting review actions to local agent workflows;

3. an embedded web application where SMEs loaded production examples, inspected proposed remediations, and supplied corrections;

4. LLM-assisted conversion from SME corrections into input/output specifications;

5. a counterexample loop that promoted selected failures into a golden dataset;

6. non-developer operation of the loop for code and prompt implementations of a complex data-cleansing pipeline.

That origin fixes the intended problem scale. The technique targets teams where the domain rule lives with an expert, the implementation lives in code and prompts, and correctness requires a durable bridge between them. The enterprise harness supplied that bridge with a UI and agent workflow. The public companion repository strips the setting down to a publishable example: a paper, clean fixtures, tests, provenance, and a small worked case.

The paper uses the enterprise setting as motivation and the public companion as evidence for the control structure. Production data, proprietary extension code, client-specific rules, and private SME workflows remain outside the public artifact. The reusable idea is the loop: SME correction → input/output spec → promoted counterexample → sketch update → agent repair → replay/compare gate.

3 Problem

Coding agents alter the economics of software change. They reduce the cost of producing a candidate patch. They also increase the number of candidate patches that look acceptable under incomplete specifications. That combination makes the specification boundary more important than the synthesis engine.

The failure mode has a recognizable shape:

1. A user gives an agent a domain task.

2. The prompt names the obvious state gap.

3. The agent finds a local patch that closes the visible gap.

4. A subject-matter expert recognizes a policy violation.

5. The violation was present in the domain, yet absent from the checkable artifacts.

The patch can be technically clean. The agent can follow local style. The tests can pass. The missing piece is the promoted counterexample: the artifact that says, "this plausible repair is forbidden for this reason, and future repairs must satisfy this rule."

3.1 Tests need intent memory

A test is a predicate over behavior. A counterexample-supplemented sketch adds a memory of intent. The distinction matters when several outputs satisfy a visible predicate. A replay check can show that a proposed row closes a state delta. A semantic compare check can show that the row used the correct operation, target, date, or status. The sketch explains why the semantic fields matter.

Tests also suffer from discoverability drift. A future agent can see a test failure and patch around it. A sketch clause linked to the test explains the policy ordering the patch must respect. That policy ordering is the thing the agent needs before it edits.

3.2 Prompts need repository anchoring

Prompts are powerful because they can describe ambiguous intent. They are weak as durable specifications because they fork easily. A prompt embedded in one agent run, notebook, or chat history can diverge from the repository's tested behavior. A sketch changes the storage location of intent: the rule lives beside the code, examples, and checks.

The method still uses prompts. It treats them as one implementation surface. Oracle B is prompt-mediated completion constrained by the same sketch and corpus as deterministic code. The prompt can carry narrative judgment; the gate keeps the result accountable.

3.3 Where formal synthesis leaves a practical gap

Formal synthesis gives the strongest result when the policy fits a formal language. Enterprise data-cleaning work often mixes formal constraints with prose conventions, client-specific policies, ambiguous notes, historical exceptions, and output formats. The method here targets that middle ground. It keeps the CEGIS loop shape and uses repository-native checks as the validator and a coding agent as the repair engine.

4 Lineage

Solar-Lezama's sketching work framed synthesis as collaboration between programmer insight and automated search: the programmer writes a partial program, leaving holes for a synthesizer to fill [5, 6]. The programmer supplies structure; the machine searches within it. CEGIS adds the validation loop: generate a candidate, validate it externally, feed counterexamples back into the next synthesis step.

Programming-by-example systems such as FlashMeta and PROSE synthesize DSL programs from examples using domain-specific deduction [4, 2]. Agent benchmarks such as SWE-bench evaluate whether language models can resolve real repository issues [3]. Tests-as-prompts work studies tests as both input and evaluation for LLM code generation [1].

Agentic synthesis against counterexample-supplemented sketches combines those traditions with ordinary repository work. The sketch resembles a partial program. The corpus resembles the growing example set in CEGIS and programming by example. The gate resembles a bounded validator. The synthesizer is a coding agent with permission to edit source and prompts.

	Sketch / CEGIS	Programming by example	This work
Human input	Partial program with holes	Examples inside a DSL	Sketch, SME corrections, counterexamples, anchors
Synthesizer	Solver-backed search	DSL learner	Coding agent editing code and prompts
Failure signal	Counterexample from validator	Missing or inconsistent example	Gate failure or SME correction
Validation	Formal or bounded decision procedure	Example consistency	Replay plus semantic compare over $\mathcal{E}$
Guarantee	Solver-relative result	DSL/example consistency	Finite-corpus correctness under repo semantics

5 Artifacts

The method is easiest to use when each artifact has one job.

Definition 1 (Sketch)**.** *A sketch $\mathcal{S}$ is a human-editable artifact that fixes the permitted strategy space and names holes. It may contain prose, tables, pseudo-code, type shapes, policy order, abstention rules, and examples of forbidden repairs. It persists in the repository and changes when a counterexample reveals missing policy.*

A sketch should answer these questions for a future agent:

1. Which operations exist?
2. Which operation has priority when several repairs close the same state gap?
3. Which fields are policy-bearing?
4. Which decisions belong in deterministic code?
5. Which decisions belong in prompt-mediated narrative completion?
6. Which cases force abstention or escalation?

Definition 2 (Corpus)**.** *The corpus $\mathcal{E} = \{(x_i, y_i)\}_{i=1}^{n}$ is the finite set of promoted examples the current gate must satisfy. Each input x_i is a concrete scenario or fixture. Each expected output y_i is the semantic repair the gate enforces.*

The corpus is authoritative because promotion is deliberate. Raw examples can be noisy. SME corrections can contain ambiguity. A promoted counterexample has passed a threshold: it names a plausible wrong output, states the rule it violated, and joins the gate.

Definition 3 (Known-code anchor)**.** *A known-code anchor $\mathcal{K}$ is an existing source artifact that constrains the agent's implementation style: an API, result type, parser convention, error shape, fixture format, test helper, reference implementation, or prompt structure. Anchors prevent parallel local inventions.*

Anchors matter because agentic repair can accidentally invent a second convention beside the first. A sketch names the policy. Anchors name the shape the codebase already uses to carry that policy.

Definition 4 (Gate)**.** *A gate $\mathcal{G}$ is the executable validation bundle that evaluates a candidate strategy $\mathcal{H}$ over every promoted case in $\mathcal{E}$. In this method the gate has two layers: replay and semantic compare.*

Replay checks whether the proposed output changes the world state correctly. Semantic compare checks whether the output used the intended rule. The two layers separate state repair from policy repair.

6 Roles

The method assigns distinct jobs to humans, agents, and checks.

6.1 Subject-matter expert

The SME supplies corrections on concrete examples through the review interface. Their review action says: this proposed output is wrong; the correct output is this; the reason is this domain rule. The system then helps convert that correction into an input/output spec.

The SME's correction has three parts:

1. the observed input;
2. the corrected output;
3. the explanation that distinguishes the corrected output from the tempting wrong one.

The explanation is crucial. It becomes the seed of a sketch update and directs the agent toward the general rule the fixture exemplifies.

6.2 Developer or maintainer

The developer curates promotion. They decide which SME corrections join $\mathcal{E}$, which sketch clauses change, and which checks enforce the new rule. They also preserve known-code anchors so the agent keeps the existing output shape.

In a mature harness, much of this curation can be assisted. The method still leaves ownership with a maintainer because promotion changes the specification.

6.3 Coding agent

The agent repairs the artifact under constraints. It may edit deterministic code, prompt text, fixtures, or tests. Its job is synthesis inside the boundary set by $\mathcal{S}$, $\mathcal{E}$, $\mathcal{K}$, and $\mathcal{G}$.

The agent should receive three forms of context before editing:

1. the sketch clause involved;
2. the counterexample and tempting wrong patch;
3. the known-code anchors that define output shape and local conventions.

6.4 Oracle A: deterministic code

Oracle A handles encodable policy. It should be boring, reviewable source code: grouping, filtering, ordering, type construction, deterministic abstention, and domain calculations. The agent can maintain it because the rule has been made concrete enough to encode.

6.5 Oracle B: prompt-mediated completion

Oracle B handles narrative or under-encoded policy: ambiguous notes, human descriptions, client-specific language, or judgment that belongs in a model prompt for the current stage of the system. Oracle B remains constrained by the sketch and checked by the same gate. A cassette can freeze its output for continuous integration.

6.6 Hybrid resolver

A hybrid resolver gives Oracle A first refusal, then routes abstentions to Oracle B. This keeps deterministic policy on the reliable path and leaves narrative completion explicit.

7 Counterexample lifecycle

The counterexample lifecycle is the core process. It turns an observed failure into a repository artifact the next agent must respect. Each stage has a separate owner so the loop keeps SME judgment, maintainer promotion, agent repair, and executable validation distinct.

7.1 Observation

A concrete case enters the review surface. It can come from production data, a synthetic fixture, a regression report, or a reviewer. The system proposes an output. The SME accepts or corrects it.

7.2 Correction

A correction names the desired output and the rule behind it. The correction should also name the tempting wrong repair when possible. This is the moment where tacit domain knowledge becomes explicit.

7.3 Spec extraction

The harness converts the correction into an input/output spec. An LLM can help draft this spec, especially when the SME explanation is prose. The maintainer or gate decides whether that draft is precise enough to promote.

7.4 Promotion

Promotion makes the case part of $\mathcal{E}$. Promotion is a specification change. The promoted case must contain enough information to reject the tempting wrong repair. A case that only records the correct answer can still leave the failure mode ambiguous.

7.5 Sketch revision

The sketch changes with the corpus. The revised sketch should state the general rule that the counterexample exposed and the fixture answer that demonstrates it. This protects the next agent pass from memorizing one row.

7.6 Repair

The agent repairs Oracle A, Oracle B, fixtures, or tests under the revised sketch. Known-code anchors constrain result shape and local convention.

7.7 Gate

The gate runs replay and semantic compare over all promoted cases. A passing gate establishes the finite-corpus result stated in Section 9.

8 Gate semantics

The gate is the executable semantics of the current corpus. It has two layers because state repair and semantic repair fail differently.

Definition 5 (Replay)**.** *Replay applies a candidate output r to an input state x and checks whether the proposed change closes the domain gap. Replay is domain-specific: it may compute balances, apply updates, recalculate derived fields, or simulate downstream effects.*

Definition 6 (Semantic compare)**.** *Semantic compare checks fields that replay leaves open: operation kind, selected entity, work date, issue type, status mutation, route, priority, or any other policy-bearing field. A row is compare-valid when it matches the promoted expectation on those fields.*

Definition 7 (Gate pass)**.** *For a strategy $\mathcal{H}$ and corpus $\mathcal{E}$, the gate passes iff every $(x_i, y_i) \in \mathcal{E}$ yields $r_i = \mathcal{H}(x_i)$ such that replay accepts (x_i, r_i) and semantic compare accepts (y_i, r_i).*

Replay catches outputs that look semantically plausible yet leave the state broken. Compare catches outputs that repair state and violate policy. A useful gate usually needs both.

8.1 Design rules for gates

A gate should be boring. It should run locally, report the exact promoted case that failed, and preserve the difference between replay failure and compare failure. The failure message should tell the next agent which rule to inspect.

Good gates have these properties:

1. **Promoted corpus coverage.** Every promoted case runs.
2. **Failure specificity.** Replay and compare failures are reported separately.
3. **Stable fixtures.** Live model calls are replaced with cassettes or recorded outputs for CI.
4. **Source links.** Each failure can be traced to sketch clause, scenario, and check.
5. **Tempting-patch rejection.** The gate includes at least one check that fails the previously plausible wrong repair.

9 Finite-corpus correctness

The proof obligation is intentionally bounded. The method proves correctness over the current promoted corpus under the repository's executable semantics. That is the useful theorem for a coding-agent workflow: when a new failure appears, it becomes a new element of $\mathcal{E}$, and the theorem's domain grows.

Definition 8 ($\mathcal{E}$-correctness)**.** *A strategy $\mathcal{H}$ is $\mathcal{E}$-correct with respect to replay function R and compare predicate C when, for every $(x_i, y_i) \in \mathcal{E}$, $R(x_i, \mathcal{H}(x_i)) = \text{true}$ and $C(y_i, \mathcal{H}(x_i)) = \text{true}$.*

Lemma 1 (Replay soundness relative to the implementation)**.** *If replay returns true for (x, r), then r satisfies the state-repair condition encoded by replay for x.*

Proof. Replay is the executable definition of state repair for the gate. It evaluates the candidate output against the input state and returns true exactly when the encoded state-repair predicate holds. Therefore a true replay result entails the state-repair condition the gate implements. □

Lemma 2 (Compare soundness relative to the corpus)**.** *If semantic compare returns true for expected output y and candidate output r, then r agrees with y on every policy-bearing field encoded by the comparer.*

Proof. The comparer enumerates the checked policy-bearing fields and fails on mismatch. Therefore a true compare result means every encoded policy-bearing field matched the promoted expectation. □

Theorem 1 (Finite-corpus gate soundness)**.** *If $\mathcal{G}$ passes for strategy $\mathcal{H}$ on corpus $\mathcal{E}$, then $\mathcal{H}$ is $\mathcal{E}$-correct with respect to the replay and compare semantics encoded by the repository.*

Proof. By definition, $\mathcal{G}$ iterates over every $(x_i, y_i) \in \mathcal{E}$, computes $r_i = \mathcal{H}(x_i)$, and requires replay and compare to pass. By replay soundness, each accepted r_i satisfies the state-repair condition encoded by replay for x_i. By compare soundness, each accepted r_i agrees with y_i on every encoded policy-bearing field. Thus every promoted corpus case satisfies both predicates, which is exactly $\mathcal{E}$-correctness. □

Scope. The theorem covers the promoted corpus, the encoded replay checker, the encoded semantic comparer, and the current golden rows. New failures expand the corpus. Checker bugs require checker fixes. Wrong golden rows require human correction. The result is narrow enough to be honest and strong enough to guide agentic repair.

10 Worked example at sketch level

RosterSynth is the paper's sketch-level worked example. It illustrates the loop through one roster remediation case. The companion repository contains implementation details for readers who want to run the example.

10.1 Scenario

A subject-matter expert sees a roster gap and two active bookings that explain it. The visible arithmetic suggests an adjustment. The policy points to duplicate cancellation. The important feature of the example is the ambiguity: two repairs can close the same apparent state gap, and only one follows the rule.

10.2 Tempting repair

The tempting repair adds an adjustment that closes the hours balance. Replay accepts the state change because the arithmetic gap disappears. Semantic compare rejects the repair because the selected operation is wrong.

10.3 Sketch rule

The counterexample promotes a sketch rule: duplicate active bookings with the same relevant shape on the pay-window end date cancel the selected duplicate when that cancellation closes the gap. The sketch now tells the next agent which operation has priority over the adjustment.

10.4 Gate lesson

The example demonstrates the paper's central gate lesson. Replay asks, "did the proposed output repair the state?" Compare asks, "did it use the rule the sketch requires?" The method needs both questions because a state-valid repair can still be a policy violation.

10.5 Artifact boundary

The paper stops at sketch level. The companion repository supplies source, fixtures, tests, recorded model outputs, and provenance for readers who want to inspect the concrete path. That split keeps the paper about the technique and the repository about evidence.

11 Applying the method

Teams can apply the method with a lightweight workflow before building a full enterprise harness. The point is to make every agent repair answer three questions: which rule is being used, which example exposed it, and which gate would reject the tempting wrong repair.

11.1 Start with the sketch

Write the intended strategy before asking the agent for code. The sketch should name the available operations, the priority order, the known holes, and the abstention cases. A weak sketch still helps because it gives failures a place to attach.

11.2 Collect examples before polishing code

Examples are useful when they distinguish competing repairs. A happy-path example shows what success looks like. A counterexample shows which plausible success should be rejected. The second kind is more valuable for agentic repair.

11.3 Name known-code anchors

Point the agent at existing result types, parser behavior, fixture format, error shape, and prompt conventions. This prevents the agent from creating a second local convention just to satisfy the current case.

11.4 Separate replay from compare

Build replay and compare as separate checks. A single assertion that says "the output looks right" hides the distinction between state repair and policy repair. Separate failures teach the next agent more.

11.5 Promote failures deliberately

Every failure should be triaged. Some failures expose fixture bugs. Some expose checker bugs. Some expose missing policy. The last category belongs in $\mathcal{E}$ and $\mathcal{S}$. Promotion is the moment where a case becomes part of the specification.

11.6 Keep generated evidence subordinate

Graphs, node files, cassettes, and extracted papers help readers inspect evidence. They support the claim after the technique is clear. The main paper should lead with the method, the theorem, and the example at sketch level; generated artifacts then answer provenance and reproducibility questions for readers who want to audit the path.

12 Companion artifact

The companion repository exists for readers who prefer examples before abstractions. It contains a clean RosterSynth slice, generated provenance, and tests that exercise the sketch-level story. The repository README gives the run commands and file map. The paper keeps implementation details in the companion artifact so the main argument stays portable across domains.

The artifact has three reader jobs:

1. let developers run the example and see the gate fail or pass;
2. let academics inspect how the finite-corpus claim maps to code;
3. let future users adapt the workflow to a different domain.

The artifact also demonstrates a practical documentation boundary. The main paper explains the technique. The README routes readers. Example docs show the concrete flow. Generated appendices expose provenance for readers who want to audit the evidence.

13 Discussion

13.1 Tests name behavior; sketches name intent

Tests can verify behavior. Counterexample-supplemented sketches add the rule the failure exposed. The failing case joins $\mathcal{E}$, and the sketch records what future agent passes must preserve.

13.2 Prompts implement one surface of the contract

The method uses prompts for narrative holes, ambiguous notes, and policy text awaiting full encoding. The sketch and corpus constrain those prompts. The gate checks their outputs. Prompting becomes one implementation surface inside the larger contract.

13.3 SMEs become specification authors

The enterprise harness changed who could add pressure to the system. SMEs corrected concrete outputs through the review interface. The harness converted those corrections into specs and candidate counterexamples, moving domain knowledge closer to the artifact that constrains future agents.

13.4 The method scales by promotion quality

The bottleneck is promotion quality. A large pile of examples gives weak guidance when the examples leave rules ambiguous. A smaller corpus of sharp counterexamples can constrain agentic repair more effectively. Promotion should therefore ask: what tempting patch does this case reject?

13.5 Generated provenance helps after the claim is clear

Graph extraction and node files help audit the path from sketch to check. They should remain supporting evidence. Readers need the method first: what is being claimed, which boundary holds, and how the example demonstrates the loop. Provenance then answers the follow-up question: where is this claim grounded?

14 Limitations

1. **Finite corpus.** The theorem covers $\mathcal{E}$. New failures must be promoted before the guarantee covers them.

2. **Checker quality.** Replay and compare inherit the limits of their encoded semantics. A buggy checker can certify the wrong behavior until a maintainer repairs the checker and replays the promoted corpus.

3. **Golden quality.** A wrong golden row makes the gate enforce the wrong behavior. Promotion therefore needs review by someone who understands the domain rule.

4. **Promotion quality.** A promoted case should distinguish a rule from a recorded answer. Weak promotion gives future agents examples to memorize; strong promotion gives them rules to preserve.

5. **Sketch quality.** Hidden prose rules still require human review. The method gives those rules a repository home, then relies on maintainers to keep that home current.

6. **Prompt drift.** Live model behavior can drift. Cassettes make CI reproducible; live behavior still needs review before a team trusts a new model response.

7. **Enterprise evidence boundary.** The public companion demonstrates the control structure on clean fixtures. The proprietary harness motivates the method and remains outside the public artifact; the public evidence supports the loop and sketches the enterprise deployment boundary.

15 Conclusion

Agentic synthesis against counterexample-supplemented sketches treats a coding agent as a synthesizer inside a constraint environment. The human writes the sketch, curates the corpus, and promotes failures. The SME corrects concrete outputs. The agent edits code and prompts. The gate supplies the finite-corpus correctness boundary.

The method turns agentic coding from a chat-centered activity into a repository-centered loop: sketch, example, correction, promotion, repair, replay, compare. That loop came from an enterprise setting where SMEs had the domain knowledge and agents had the editing surface. The reusable technique is the bridge between them. Each promoted counterexample makes the same wrong patch harder to repeat, and each passing gate states exactly what has been proven: correctness for the current promoted corpus under the replay and semantic comparison semantics the repository encodes.

References

[1] Yi Cui. Tests as prompt: A test-driven-development benchmark for llm code generation, 2025.

[2] Sumit Gulwani, Oleksandr Polozov, and Rishabh Singh. Program synthesis. *Foundations and Trends in Programming Languages*, 4(1–2):1–119, 2017.

[3] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R. Narasimhan. SWE-bench: Can language models resolve real-world github issues? In *The Twelfth International Conference on Learning Representations*, 2024.

[4] Oleksandr Polozov and Sumit Gulwani. Flashmeta: A framework for inductive program synthesis. In *Proceedings of the 2015 ACM SIGPLAN International Conference on Object-Oriented Programming, Systems, Languages, and Applications*, OOPSLA 2015, pages 107–126. ACM, 2015.

[5] Armando Solar Lezama. *Program Synthesis by Sketching*. PhD thesis, EECS Department, University of California, Berkeley, December 2008.

[6] Armando Solar-Lezama. Program sketching. *International Journal on Software Tools for Technology Transfer*, 15(5–6):475–495, 2013.